Project Brief: Interactive Internal Knowledge Base (RAG Bot)

Type: Collaborative End-of-Semester Project
Line Spacing: 1.25

1. Project Overview

The goal of this project is to build an interactive, web-based Internal Knowledge Base powered by Retrieval-Augmented Generation (RAG). Instead of manually searching through static documents, staff members should be able to upload company PDFs, text files, or Markdown guides, and use a conversational chat interface to ask questions and get answers grounded directly in those uploaded documents.
This project is structured specifically to blend deep artificial intelligence execution with robust web application development and workflow integration. It simulates a modern corporate development environment where a backend/AI layer must seamlessly communicate with a frontend user interface.

2. Team Roles & Responsibilities

Role	Core Responsibilities & Technical Focus
AI & Systems Engineer (The Infrastructure Specialist)	● Set up the core Retrieval-Augmented Generation (RAG) pipeline. ● Handle document processing: parsing PDFs/Markdown, text chunking strategies, and embedding generation. ● Implement and manage the vector database storage. ● Orchestrate the Large Language Model (LLM) interaction to enforce context grounding and eliminate hallucinations. ● Build clean RESTful API endpoints for the frontend to consume.
Application & Integration Developer (The Product Specialist)	● Design and build the complete frontend user experience and web interface. ● Create the document upload management system and file tracking views. ● Develop a highly responsive, stateful chat interface for querying the knowledge base. ● Connect the frontend views to the AI endpoints using async network requests. ● Implement visual "source citation" elements to display the

Role	Core Responsibilities & Technical Focus
	document chunks used by the AI.

3. Technical Architecture & Constraints

To avoid blocking each other during development, you must adhere to a strict separation of concerns. The application will be split into two primary layers:

- **The Frontend Layer:** Handles user interactions, state management, file upload forms, and the chat history timeline. It interacts with the backend strictly via JSON HTTP requests.
- **The Backend/AI Layer:** Handles long-running file processing, text embedding execution, vector search retrieval, and LLM communication. It exposes clean REST API endpoints.

Critical Integration Requirement: On Day 1, both developers must collaboratively design and agree upon the API schema (the exact JSON formats for uploading files and transmitting chat payloads). Once this contract is set, both layers can be built in parallel.

4. Project Milestones & Deliverables

The project must be executed in accordance with the following four chronological milestones. Each milestone requires a functional, working code demonstration.

Milestone 1: API Contract & Architecture Setup (Target: Week 1-2)

- **AI Specialist Deliverable:** Initialize the backend repository framework (e.g., Python/FastAPI or preferred stack) and scaffold mock API endpoints for file uploads and text queries that return static placeholder JSON data.
- **Application Specialist Deliverable:** Scaffold the web application structure (repository setup, routing, and primary page layouts). Implement placeholder components for the document list and the chat container.
- **Shared Checkpoint:** Successfully trigger a network request from the frontend interface to the backend server and log the mock JSON response in the UI console.

Milestone 2: Data Ingestion & Storage Pipeline (Target: Week 3-4)

- **AI Specialist Deliverables:**
    - Implement a file processing service that extracts raw text from uploaded PDFs or text files.
    - Create a chunking script to break text into logical segments (e.g., 500-token blocks with overlap).
    - Integrate an embedding model to convert text chunks into vectors and save them into a vector database.
- **Application Specialist Deliverables:**

    - Build the functional Document Management UI, featuring file drag-and-drop capabilities.
    - Connect the upload form to the backend multipart file upload endpoint.
    - Create a dashboard view displaying a list of successfully ingested files and their indexing statuses.
- **Shared Checkpoint:** Upload a real multi-page PDF through the frontend dashboard and verify via backend logs or vector db inspection that the text was successfully chunked, embedded, and stored.

Milestone 3: Core Retrieval & Conversational Chat (Target: Week 5-6)

- **AI Specialist Deliverables:**
    - Build a semantic search engine that takes an incoming user query, generates its vector embedding, and retrieves the top-K matching document chunks from the vector database.
    - Construct a strict prompt template that forces the LLM to answer the user query *only* using the retrieved chunks.
    - Update the query endpoint to return the LLM's text response along with metadata detailing the source document names and exact text chunks used.
- **Application Specialist Deliverables:**
    - Transform the chat placeholder into a dynamic, stateful conversational interface.
    - Handle user message submissions, loading states while waiting for the API response, and appending incoming AI replies.
    - Parse the metadata returned by the backend to render clean, clickable "Source Citations" beneath the AI's response text.
- **Shared Checkpoint:** Ask the chat interface a specific question contained only within an uploaded document. The UI must display a coherent, context-grounded answer accompanied by the correct file citation.

Milestone 4: Fine-Tuning, Error Handling & Polish (Target: Week 7-8)

- **AI Specialist Deliverable:** Refine prompt engineering to handle edge cases (e.g., instruct the model to politely state "I cannot find this in the database" if no relevant chunks are found, preventing hallucinations). Optimize search parameters (Top-K tuning).
- **Application Specialist Deliverable:** Add robust UI error handling for network timeouts or failed uploads. Optimize chat scrolling behaviors and add clean user feedback animations (such as typing indicators).
- **Shared Checkpoint:** Conduct complete end-to-end testing. Prepare the final codebase, ensure comprehensive documentation is added to the repositories, and finalize the presentation slides for the end-of-semester assessment.

5. Engineering Best Practices

To successfully pass the final technical evaluation, the project must demonstrate high standards of professional collaboration:

1. **Version Control via Git:** All source code must reside in version-controlled repositories.

Work must be tracked using feature branches. Direct commits to the main branch are prohibited; all additions must be introduced via Pull Requests or Merge Requests.

2. **Interface Decoupling:** The backend must remain completely agnostic of the frontend design. The frontend must remain agnostic of the specific LLM or vector database engine chosen by the AI specialist.
3. **Documentation:** Both repositories must include an explicit README.md file detailing installation procedures, environmental dependencies, configuration instructions, and a quick-start guide to spin up the local development environment.